

## myCompiler 擴充功能

### 一、型別系統與資料結構

| 功能                                 | 文件說明（含功能簡述）                                                                                                              | 實際狀態 / 測試檔案                                                                                  |
|------------------------------------|--------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|
| double 型別                          | 支援 64-bit 雙精度浮點數常數解析與常數折疊運算。                                                                                             | test_advanced_types.c, test_fd.c, test_type_system.c                                         |
| bool / true / false                | 支援標準布林型別，底層映射至 LLVM i1 核心型別。                                                                                             | test_advanced_types.c, test_type_system.c                                                    |
| 標準布林型別 (Boolean) 語意與隱式升級機制         | 實作 C99 規範的 bool 型別處理。攔截非零整數至布林值的賦值。並支援 bool 型別在多種複雜情境（包含數學運算式、函式引數傳遞、陣列賦值、以及向上擴充至 long 等大字節型別）下的隱式整數升級 (Zero-extension)。 | test_bool_bug.c, test_bool_bug2.c                                                            |
| 原生布林型別 (_Bool)                     | 於 AST 解析階段的 typeSpecifier 新增 _Bool 專屬分支，語意與行為等同於標準 bool，底層映射至 LLVM 的 i1 核心型別。                                            | test_new_features_v4.c                                                                       |
| 型別修飾詞透傳與靜默處理 (volatile / restrict) | 擴充變數與指標宣告之修飾詞支援。於 typeSpecifier 新增前綴透傳分支，解析時靜默承認並吸收 volatile 與 restrict 關鍵字，隨後將內層真實型別正確傳遞給符號表，避免未定義關鍵字造成的語法錯誤。           | test_new_features_v4.c                                                                       |
| 一維陣列 + float 陣列初始化清單               | 支援大括號 {} 初始賦值(Ex: int arr[5] = {10, 20, 30, 40, 50};)與記憶體線性配置，自動計算偏移量。可支援陣列元素修改、加總。                                      | test_new_features.c, test5.c, test7.c, test_array.c, test_all_features.c, test_type_system.c |
| 二維陣列 int a[3][4]                   | 支援嵌套陣列結構 [3 x [4 x i32]]，藉由雙重 GEP 實現二維偏移讀寫，支援宣告與存取、陣列走訪。                                                                 | test_feat4.c, test_array.c, test_all_features.c, test_type_system.c                          |
| 二維陣列函式參數傳遞                         | 支援將多維陣列傳遞至函式中（如 int arr[][4]）。編譯器能正確計算內層維度的記憶體 Stride 與偏移量，並進行陣列元素的讀寫、走訪與數學運算。                                           | test_v6_features.c                                                                           |
| 三維陣列 (3D Array) 與多層 GEP 指標運算       | 延伸陣列型別系統的維度支援至三維（如 int grid[2][3][4]）。支援 3D 陣列                                                                           | test_v7_features.c                                                                           |

|                               |                                                                                                                                                                                  |                                                                                                                                        |
|-------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
|                               | <p>的宣告、巢狀迴圈初始化及作為函式參數傳遞。編譯器後端能處理多層維度的 <b>Stride</b> 計算，並生成包含 4 個索引參數的 <b>getelementptr LLVM</b> 指令。</p>                                                                          |                                                                                                                                        |
| char 純量、char 陣列、字串字面值初始化      | <p>支援字元與字串常數，傳遞至函式時自動退化為 <b>i8*</b> 指標，<b>char</b> 陣列可參與整數運算。</p>                                                                                                                | <p>test_advanced_features.c,<br/>test_char_recursion.c,<br/>test_advanced_types.c,<br/>test_all_features.c,<br/>test_type_system.c</p> |
| 16 進位 (0x) 與 2 進位 (0b) 字面值支援  | <p>詞法階段自動解析 <b>0x</b> 與 <b>0b</b> 開頭之整數常數，並轉換為十進位字串。</p>                                                                                                                         | <p>test_bitwise.c,<br/>test_advanced_suite.c</p>                                                                                       |
| 字串字面值直接賦值 Ex: s = "hello"     | <p>支援將字串常數指標直接賦值給字元指標或陣列。</p>                                                                                                                                                    | <p>test_array.c, test_new5.c,<br/>test_type_system.c</p>                                                                               |
| 字串與字元跳脫序列解析擴充                 | <p>強化 <b>Lexer</b> 與字串常數產生器。支援十六進位 (<b>\xNN</b>) 與八進位 (<b>\ooo</b>) 的 ASCII 碼跳脫序列，並正確轉換為 <b>LLVM IR</b> 的位元組表示法。</p>                                                             | <p>test_v8_features.c</p>                                                                                                              |
| struct / typedef struct       | <p>支援自訂結構體定義、別名綁定，以及藉由 <b>.</b> 運算子計算成員位址存取。</p>                                                                                                                                 | <p>test_struct.c,<br/>test_all_features.c,,<br/>test_final_features.c,<br/>test_type_system.c</p>                                      |
| typeof 泛型推導與 Dry-Run 模式       | <p>支援 <b>GNU C</b> 擴充的 <b>typeof(expr)</b> 語法。於 <b>AST</b> 解析階段實作「<b>Dry-Run</b> 模式」，在推導複雜運算式型別的同時，阻斷 <b>LLVM IR</b> 的副作用生成。支援泛型巨集 (如 <b>SWAP</b>) 的多型別動態替換。</p>                 | <p>test_v8_features.c</p>                                                                                                              |
| C99 複合字面值 (Compound Literals) | <p>支援 <b>(type){initializer}</b> 語法。允許於執行期表達式中，動態宣告並初始化匿名陣列或結構體純量，並處理其記憶體配置與指標退化。</p>                                                                                            | <p>test_v8_features.c</p>                                                                                                              |
| 記憶體對齊 <b>_Alignof</b> 運算子     | <p>支援 <b>_Alignof</b> 關鍵字。編譯器前端能判斷純量與複合資料型別在 <b>x86_64</b> 目標架構下的底層記憶體對齊規範。</p>                                                                                                  | <p>test_v8_features.c</p>                                                                                                              |
| 結構體與陣列具名初始化                   | <p>支援 <b>C99</b> 標準的具名初始化語法。允許於大括號內指定 <b>struct</b> 特定欄位賦值（如 <b>{.x=10, .y=20}</b>），或針對陣列進行跳躍式特定索引賦值（如 <b>{[2]=9, [4]=42}</b>）。底層實作 <b>Zeroinitializer</b> 機制，能自動將未指定的欄位或陣列元</p> | <p>test_v7_features.c</p>                                                                                                              |

|                                     |                                                                                                                                                          |                                                                             |
|-------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------|
|                                     | 素初始化為 0。                                                                                                                                                 |                                                                             |
| 聯合體 (Union)                         | 多個變數「共用同一塊記憶體位址」。改變其中一個成員，其他成員的值也會跟著記憶體位址的位元改變。                                                                                                          | test_union.c                                                                |
| 匿名結構體與聯合體 (Anonymous Struct/Union)  | 支援 C11 標準的匿名 struct 與 union 巢狀宣告。編譯器前端能自動扁平化成員存取路徑，允許跨層級直接存取內部欄位 (例如直接使用 v.x 或 col.r)，並於後端處理 Little-Endian 架構下的記憶體共用與佈局配置。                               | test_anon_struct.c                                                          |
| enum (含自訂值)                         | 支援列舉型別定義，可手動指定或由編譯器自動遞增常數值。可當變數使用、在 if-then-else、支援位元計算。                                                                                                 | test_feat6.c, test_enum.c, test_all_features.c,, test_type_system.c         |
| typedef 型別別名                        | 支援建立既有型別的自訂別名綁定 (映射至符號表別名對照表) (可多層別名)。                                                                                                                   | test_typedef.c                                                              |
| 函式指標別名宣告 (Function Pointer Typedef) | 支援解析如 typedef int (*Cmp)(int, int); 之函式指標別名宣告。於 AST 解析並將其型別資訊註冊至符號表，使後續程式碼能以此別名宣告變數並順利進行函式呼叫。                                                            | test_v7_features.c                                                          |
| 結構體位元欄位 (Bit-field) 記憶體佈局與存取        | 支援結構體內以位元為單位的欄位宣告 (如 unsigned int flags : 4;)。後端能計算記憶體佈局，將多個位元欄位壓縮於同一整數空間內；並於變數讀寫時，自動發射底層的 Load、位移 (Shift)、遮罩 AND/OR (Masking) 及 Store 的 LLVM IR 複雜指令組合。 | test_v7_features.c                                                          |
| sizeof(type) / sizeof(expr)         | 在編譯期動態計算並折疊型別或運算式之位元組大小 (1, 4, 8 bytes)。                                                                                                                 | test_feat6.c, test_sizeof.c, test_type_system.c, test_extended_types_full.c |
| long / long long 型別                 | 支援 64-bit 有號長整數的宣告、算術運算與大規模數值比較。在與 32-bit 常數運算時，編譯器會自動安插 sext 進行隱式型別提升。                                                                                  | test_long.c                                                                 |
| 變數長度陣列 (VLA)                        | 陣列大小由執行期變數動態決定。編譯器會在執行期動態計算長度並轉換為 i64，發射動態 alloca 記憶體配置，                                                                                                 | test_vla.c                                                                  |

|                                          |                                                                                                                                                                                                            |                            |
|------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------|
|                                          | 並完美整合 Varargs 的浮點數升級 (Float Promotion) 傳參數。                                                                                                                                                                |                            |
| 標量型別 (Short / Unsigned)                  | 擴充 TypeInfo 與 typeSpecifier，支援 short, signed/unsigned char, unsigned short, unsigned int, unsigned long。實作編譯期 sizeof 與記憶體對齊規則 (Short/UnsignedShort=2, UnsignedChar=1)，底層精準映射至 LLVM 的 i8 / i16 / i32 / i64。 | test_extended_types_full.c |
| stdint.h 固定寬度型別                          | 擴充編譯器型別表 (typedefMap)，支援 int8_t 到 int64_t、對應的無號 uint 系列以及 size_t。並修復大整數解析機制，採用 Long.parseLong 保留 64-bit 常數折疊精度。(int8_t, uint8_t, int16_t, uint16_t, int32_t, uint32_t, int64_t)                            | test_new4features.c        |
| 無號指令選擇與整數提升 (Promotion)                  | 新增 isUnsigned 旗標。算術與 % 運算自動發射無號/長整數專用指令 (udiv, urem, srem, lshr, icmp u* 系列)。實作 C 語言隱式轉型機制，調用 sext、zext 與 trunc 處理跨寬度變數賦值，並支援 printf 可變參數中 Short 型別的 zext 強制擴充。                                            | test_extended_types_full.c |
| GNU 陣列範圍初始化 (Array Range Initialization) | 擴充 C99 具名初始化功能，支援 GNU C 標準的陣列範圍賦值語法 [lo ... hi] = val。允許開發者以極精簡的語法為一段連續索引賦予相同初始值。解析器能精準計算起訖範圍並自動展開賦值，相容與單點具名初始化、一般順序初始化的混合使用，並自動處理未指定元素的補零及內部索引計數器的接續。                                                     | test_range_init.c          |
| C99 彈性陣列成員                               | 支援 C99 標準之彈性陣列成員。允許結 Struct 的最後一個欄位宣告為未定長度的陣列 (如 char data[];)。配合動態記憶體配置 (malloc(sizeof(struct) + N)) 實現連續且零額外開銷的變長記憶體佈局。                                                                                  | test_v9_features.c         |
| POSIX 與系統底層型別相容性                         | 支援解析與處理標準 C 及 POSIX 規範的底層型別 (包含 time_t, clock_t, ptrdiff_t, ssize_t, off_t)。前端識別並                                                                                                                          | test_missing_types.c       |

|  |                                                                                     |  |
|--|-------------------------------------------------------------------------------------|--|
|  | 映射至 LLVM 64-bit 架構大小；同時支援以 typedef 封裝 FILE* 檔案串流與 va_list 可變參數指標，提升對作業系統標頭檔的編譯相容能力。 |  |
|--|-------------------------------------------------------------------------------------|--|

## 二、變數宣告、作用域

| 功能                       | 文件說明（含功能簡述）                                                                            | 實際狀態 / 測試檔案                                                                                             |
|--------------------------|----------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|
| 多變數同行宣告（含初始值）            | 支援如 <code>int a=1, b=2;</code> 的連鎖宣告，並支援即時常數折疊。                                        | test_feat6.c,<br>test_advanced_types.c,<br>test_scope.c                                                 |
| 全域變數/區域變數符號表 + Shadowing | 讀寫、float 型別、陣列等。並支援多層級 Block Scope 作用域，內部區塊同名變數可遮蔽外部變數。                                | test5.c, test_feat6.c,<br>test_global_scope.c,<br>test_all_features.c,<br>test_scope.c                  |
| for 迴圈變數 scope 隔離        | <code>for(int i=0;...)</code> 中的變數 i 隔離於獨立作用域，僅在該迴圈生命週期內有效。                            | test_advanced_suite.c,<br>test_global_scope.c,<br>test_feat6.c,<br>test_scope.c,<br>test_control_flow.c |
| const 變數                 | 標記唯讀常數變數，若事後發生寫入或賦值企圖則於編譯期攔截並報錯。                                                       | test_feat4.c,<br>test_const.c,<br>test_scope.c                                                          |
| static 局部變數              | 支援函式內宣告 static 局部變數。確保變數生命週期跨越函式呼叫並保持狀態，底層將其映射至 LLVM IR 的內部全域變數，同時在符號表中正確限制其作用域僅在該函式內。 | test_v6_features.c                                                                                      |
| 語意防禦機制                   | 發生重複宣告或型別錯置時進行動態修補與防護，防止引發 LLVM 後端崩潰。                                                  | test_boundary.c,<br>test_scope.c,<br>test_semantic.c,<br>test_pointer.c                                 |

## 三、控制流與作用域

| 功能                                       | 文件說明（含功能簡述）                     | 實際狀態 / 測試檔案                                                             |
|------------------------------------------|---------------------------------|-------------------------------------------------------------------------|
| while / do-while / for（可多層 Nested Loops） | 支援標準三大迴圈之控制流 IR 生成（條件判定與迴圈體跳轉）。 | test_rubric.c, test2.c,<br>test_control_flow.c,<br>test_all_features.c, |

|                                              |                                                                                                                                                                                     |                                                                                                                                                                                                                                                    |
|----------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                              |                                                                                                                                                                                     | ultimate_test.c,<br>test_final_features.c                                                                                                                                                                                                          |
| 巢狀迴圈混用                                       | 支援多層迴圈相互嵌套，例如 for 包 while。正確維護並隔離各自的條件與跳轉 Basic Block。                                                                                                                              | test2.c, test7.c,<br>test_all_features.c,<br>test_control_flow.c,<br>test_rubric.c, test_arch.c                                                                                                                                                    |
| switch-case                                  | 支援 switch、case 與 default 語法。不同於單純的 if-else 巢狀展開，編譯器底層直接對應並發射 LLVM 原生的 switch 指令，利用 Jump Table 機制達成 O(1) 時間複雜度的高效跳轉，並支援區塊執行與 fall-through 語意。產出 LLVM switch i32 跳轉表指令，支援 default 分支。 | test5.c, test7.c, test8.c,<br>test_control_flow.c,<br>test_all_features.c                                                                                                                                                                          |
| switch-case Fall-through                     | 支援 switch 條件控制流，包含標準 C 語言預設的 Fall-through 行為（若無 break 指令則無條件繼續執行下一個 case 的區塊），以及 default 分支處理。                                                                                      | test_v6_features.c                                                                                                                                                                                                                                 |
| break / continue                             | 透過迴圈標籤堆疊（Loop Label Stack）管理跳轉目標，確保巢狀結構中短路控制。                                                                                                                                       | test_new4.c, test7.c,<br>test8.c,<br>test_control_flow.c,<br>test_all_features.c                                                                                                                                                                   |
| 支援多層 if-else（Nested If-Else）、while+if。       | 每當遇到一個新的 { (無論是 if, while, 還是 for)，呼叫 pushBuffer() 等到遇到 } 結束時，呼叫 popBuffer()。並支援 while+if。                                                                                          | test_control_flow.c,<br>test_strict_80.c,<br>test_rubric.c,<br>ultimate_test.c,<br>trap_test.c, test9.c, test2.c,<br>test_all.c, test_arch.c,<br>test_final_features.c,<br>test_goto.c,<br>test_local_fp.c,<br>test_new4.c,<br>test_ptr_features.c |
| goto 與標籤陳述式                                  | 支援 C 語言無條件跳轉語意，包含往前跳轉、往後跳轉模擬迴圈、跳出多重巢狀迴圈，以及多重標籤跳轉。底層映射至 LLVM IR 的 Basic Block 切割與 br label 指令。                                                                                       | test_goto.c                                                                                                                                                                                                                                        |
| for 迴圈多變數宣告 (C99 For-Loop Multi-Declaration) | 支援 C99 標準的 for 迴圈初始化區塊多變數宣告機制（例如 for (int i = 0, j = 10; ...)）。解析器能正確處理逗號分隔                                                                                                         | test_batch1.c                                                                                                                                                                                                                                      |

|                                                           |                                                                                                                                                                                                                                        |                                                                                                                     |
|-----------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
|                                                           | 的同型別宣告，於該迴圈的局部作用域內正確配置 <code>alloca</code> 記憶體，並在離開迴圈後安全隔離，提升程式碼的簡潔度。                                                                                                                                                                  |                                                                                                                     |
| 陣列雙重邊界防護 (Bounds Checking)                                | 實作高階記憶體安全機制，提供動靜態雙重防護：<br><br>1. 靜態分析 (Static)：結合常數折疊技術，針對編譯期已知的常數索引進行越界計算，若越界則發出 <b>Warning</b> 警告 (容錯不中斷編譯)。<br><br>2. 動態攔截 (Dynamic)：於 LLVM IR 動態注入驗證指令，若執行期發生變數索引越界，則印出詳細錯誤資訊並觸發 <code>abort()</code> ，防止 <b>Buffer Overflow</b> 。 | <code>test_bounds_static_warn.c</code> ,<br><code>test_bounds_dynamic.c</code> ,<br><code>test_v8_features.c</code> |
| 控制流防呆：遺漏回傳值警告 (Missing Return)                            | 於 AST 走訪階段掃描非 <code>void</code> 函式的 <b>Control Flow</b> 。若偵測到所有執行路徑未完全包含 <code>return</code> 敘述，將於編譯期發出 <b>Warning</b> ，並於後端自動補上預設的 <code>ret</code> 指令防護網，避免觸發未定義行為。                                                                  | <code>test_v8_features.c</code>                                                                                     |
| 非區域跳轉與例外處理 ( <code>setjmp</code> / <code>longjmp</code> ) | 實作跨函式的非區域跳轉 (Non-local Goto) 控制流。支援 <code>jmp_buf</code> 型別變數儲存當前執行環境，對接底層系統庫的 <code>setjmp</code> 與 <code>longjmp</code> 呼叫。突破常規函式的 <b>Call Stack</b> 限制，賦予 C 編譯器模擬 <b>Exception Handling</b> 與錯誤恢復的能力。                               | <code>test_v9_features.c</code>                                                                                     |

## 四、運算子

| 功能                                       | 文件說明（含功能簡述）                                                                                            | 實際狀態 / 測試檔案                                                                                                                                                                                                                   |
|------------------------------------------|--------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 三元運算子 <code>?:</code> （可多層 Nested、float） | 支援 <code>cond ? true_val : false_val</code> 的單行條件回傳語意。依條件評估結果安插 <code>br</code> 分支，動態選擇性執行對應之左側或右側運算式。 | <code>test_advanced_features.c</code> ,<br><code>test_new_features.c</code> ,<br><code>test_control_flow.c</code> ,<br><code>test_operators.c</code> ,<br><code>test_all_features.c</code> ,<br><code>test_operators_c</code> |
| 算術與複合賦值                                  | 支援 <code>%</code> 餘數運算、 <code>++/--</code> 遞增減，以及 <code>+=, -=, *=, /=, %=</code> 等複合運算。               | <code>test_operators.c</code> ,<br><code>test_operators_c</code> ,                                                                                                                                                            |

|                                                               |                                                                                                                   |                                                                                                                     |
|---------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
|                                                               |                                                                                                                   | test_all_features.c,<br>test_rubric.c                                                                               |
| 邏輯運算與短路求值                                                     | 支援 &&,   , !，並實作「左側條件若已成立即不執行右側」的短路評估優化。(用「Basic Block 跳轉」來實作，而不是單純用 and/or 指令)                                   | test6.c,<br>test_all_features.c,<br>test_operators.c,<br>test_operators_.c,<br>test_strict_80.c,<br>ultimate_test.c |
| 位元運算                                                          | 支援 &,  , ^, ~, <<, >> 完整的二進位位元指令運算，可複合應用。                                                                         | test_bitwise.c,<br>test_operators.c,<br>test_operators_.c,<br>test_strict_80.c                                      |
| 位元複合賦值                                                        | 定義並支援 &=,  =, ^=, <<=, >>= 這些進階的位元複合賦值運算。                                                                         | test_advanced_suite.c                                                                                               |
| 關係運算式之隱式整數擴充與無分支運算支援 (Zero-Extension & Branchless Evaluation) | 突破基礎編譯器僅將關係運算子（如 ==, <）用於條件跳轉之限制。實作 C 語言標準，使關係運算結果能作為一等表達式參與數學運算。底層實作 LLVM 零擴充指令 (zext)，將 1-bit 布林值拉伸為 32-bit 整數。 | test_bool_math.c                                                                                                    |

## 五、編譯期優化

| 功能                           | 文件說明（含功能簡述）                                                                                                                                         | 實際狀態 / 測試檔案                                                               |
|------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------|
| 常數折疊（Constant Folding）       | [前端 AST 優化] 實作基礎的編譯期最佳化機制。在 AST 遍歷階段，若偵測到純常數的四則運算或取餘數運算（例如 24 * 60 * 60），編譯器會直接在編譯期計算出結果，僅生成單一常數的 LLVM IR，大幅降低執行期的運算開銷。                             | test_advanced_features.c,<br>test_new_features.c,<br>test_optimizations.c |
| 分支預測最佳化指令 (__builtin_expect) | [後端 LLVM 引導] 導入相容於 GCC/Clang 體系的底層效能最佳化內建函式 (Built-in function)。允許開發者在編譯期提供 Branch Prediction Hint。透過此擴充，編譯器能引導 LLVM 產生最佳化的條件跳轉組合，減少現代 CPU 的管線停滯機率。 | test_batch1.c                                                             |
| 編譯器特徵函式靜默與展開                 | 擴充預處理器與 AST 解析，支援 GCC/Clang 體系的高階特徵函式。針對                                                                                                            | test_batch1.c                                                             |



|                                                                            |                                                                                                                                              |                                                                                                                                                                        |
|----------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ( <code>__builtin_constant_p</code> / <code>__builtin_unreachable</code> ) | <code>__builtin_constant_p</code> 進行編譯期常數靜默判定（展開為 0）；針對 <code>__builtin_unreachable</code> 展開為 (void)0 作為無副作用之空敘述。相容系統標頭檔，並提供 DCE 與分支優化的合法提示 |                                                                                                                                                                        |
| GNU C 擴充屬性 ( <code>__attribute__</code> ) 靜默處理                             | AST 解析階段擴充語法樹，支援並「靜默吸收」GNU C 的 <code>__attribute__((...))</code> 語法。確保在編譯含有此類系統級屬性的標準標頭檔或原始碼時，不會觸發 Syntax Error。                             | test_v6_features.c                                                                                                                                                     |
| 代數化簡                                                                       | 移除無效代數運算（如加上 0、減掉 0、乘以 1、乘以 0 之分支優化與化簡）。                                                                                                     | test_rubric.c,<br>test_optimizations.c                                                                                                                                 |
| CSE（公共子式消除）                                                                | 辨識並複用重複的相同運算式（透過雜湊表紀錄），避免重複產生冗餘 IR。                                                                                                          | test_new5.c,<br>test_optimizations.c                                                                                                                                   |
| 前後端 DCE（Dead Code Elimination）                                             | 移除不可達代碼（UCE）、無效儲存（DSE）與未使用的純計算賦值（TDA）。                                                                                                       | test_dce.c,<br>test_3optimizations.c,<br>test_optimizations.c,<br>test_advanced_types.c,<br>test_all_features.c,<br>test_new4.c,<br>test_new5features.c,<br>test_opt.c |
| 短路求值                                                                       | 邏輯運算子動態跳轉優化，提早終止不必要的後續條件評估。                                                                                                                  | test_3optimizations.c,<br>test_optimizations.c                                                                                                                         |
| Strength Reduction                                                         | 將乘除以 2 的幕次運算在編譯期自動化簡為更高效的位元左移/右移指令。                                                                                                          | test_3optimizations.c,<br>test_advanced_types.c,<br>test_new5features.c,<br>test_opt.c                                                                                 |
| 動態型別強度削減 (Dynamic Strength Reduction)                                      | 進階強化乘除法強度削減功能，支援 32 位元 (i32) 與 64 位元 (i64) 的動態型別推導。無論是 int 還是 size_t，皆能自動判斷並發射對應位元寬度的位移指令 (shl, ashr, lshr)。                                 | test_new4features.c,<br>test_opt.c                                                                                                                                     |
| 隱式與顯式型別轉換 explicit cast、implicit cast                                      | 自動處理型別升降級（int/float/double/char/(long) long），也可以函式參數隱式轉換；支援強制顯式轉型並確保負數等截斷方向正                                                                 | test_advanced_features.c,<br>test_all.c, test3.c, test6.c,<br>test_type_conversion.c,<br>test_all_features.c,<br>test_char_recursion.c,                                |

|                                           |                                                                                                                                                                      |                                                                                                                                 |
|-------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|
|                                           | 確。                                                                                                                                                                   | test_rubric.c, ultimate_test.c,<br>trap_test.c<br>test_vla.c<br>test_long.c,<br>test_extended_types_full.c                      |
| DCE 逃逸分析                                  | 靜態分析並追蹤指標位址是否外流，確保被外部使用的變數不會被 DCE 誤刪。                                                                                                                                | test_dce.c,<br>test_3optimizations.c,<br>test_optimizations.c,<br>test_advanced_types.c,<br>test_all_features.c,<br>test_new4.c |
| 迴圈不變式外提 (LICM)                            | 自動識別 while 與 for 迴圈中與疊代無關的 loop-invariant 運算，提至 Preheader 區塊。支援強度削減 (將乘 2 優化為左移 shl) 與自動型別提升外提。                                                                      | test_licm.c                                                                                                                     |
| 窺孔最佳化 (Peephole Optimization) 與強度削減       | 完整實作代數恆等式化簡 (如消除 $x+0$ , $x*1$ , $x*0$ )。內建強度削減 (Strength Reduction)，能將乘以/除以 2 的次方自動優化為 CPU 執行效率極高的位元移位 (shl, ash) 指令                                                | test_peephole.c                                                                                                                 |
| 常數分支壓平 (Constant Branch Flattening)       | 在 AST 遍歷與 IR 發射階段，能自動評估條件式是否為編譯期常數 (例如 if (1) 或 if (0))。直接消除不必要的 branch 分支跳轉，並對永遠無法抵達的區塊 (Unreachable Block) 執行強制的死碼消除 (DCE)。                                        | test_peephole.c                                                                                                                 |
| 全域常數傳播 (Global Constant Propagation, GCP) | 實作進階資料流分析。編譯器能跨 Basic Blocks 追蹤常數賦值，將後續的變數讀取 (Load) 自動替換為常數，並觸發連鎖的窺孔優化 (Peephole)。具備逃逸與修改偵測機制，遇迴圈或函式呼叫時會保守停止傳播，確保優化安全性。                                              | test_gcp.c                                                                                                                      |
| LLVM 底層內建函式 (Built-in Intrinsics)         | 支援 GCC/Clang 體系的 __builtin_* 編譯器內建函式。包含 popcount (位元 1 個數)、clz (前導零)、ctz (尾隨零) 與 bswap (位元組反轉)，及其 64-bit 擴充版本 (*l)。直接對接 LLVM 底層的高效硬體指令，可極速實現 log2 取整或判定 2 的幕次等進階演算法。 | test_builtins.c                                                                                                                 |
| 迴圈展開優化 (Loop                              | 針對已知迭代次數之小型迴圈 (如                                                                                                                                                     | test_v9_features.c                                                                                                              |

|                                     |                                                                                                                                                                                             |            |
|-------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------|
| Unrolling)                          | for(int i=0; i<4; i++) )，編譯器前端能預先模擬並自動展開迴圈體，直接消除 LLVM IR 層級的條件判斷 (cond) 與跳轉 (br) 指令開銷。內建展開門檻防護（防範程式碼膨脹 Code Bloat）及動態 Step 計算能力。                                                            |            |
| 尾呼叫優化 (Tail Call Optimization, TCO) | 遞迴與記憶體最佳化機制。於 AST 解析與 IR 發射階段，識別處於函式絕對尾端 (Tail position) 的函式呼叫與自我遞迴。編譯器會自動發射 LLVM 的 tail call 指令，允許底層重複利用當前的 Stack Frame，將 O(N) 空間複雜度的呼叫堆疊消耗強制壓縮為 O(1) 的常數空間，解決深層遞迴造成的 Stack Overflow 崩潰問題。 | test_tco.c |

## 六、函式系統

| 功能                        | 文件說明（含功能簡述）                                                                                         | 實際狀態 / 測試檔案                                                                                               |
|---------------------------|-----------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| 自訂函式、遞迴、多參數               | 支援標準函式定義、多重參數堆疊傳遞與自我遞迴呼叫。另外，遞迴可互相呼叫(test_all_features.c)。                                           | test4.c, test_prototype.c, test_recursion.c, test_char_recursion.c, test_all_features.c, test_functions.c |
| 前置宣告（Prototype）           | 允許在函式完整實作前進行簽名宣告，支援函式的前向引用（Forward Reference）。                                                      | test_prototype.c, test_functions.c, test_all_features.c                                                   |
| char 整數提升                 | 嚴格遵守 C 標準（Strict Integer Promotion），算術運算前將 char 自動提升為 32-bit int。<br>Ex: char next; next = ch + 1;a | test_advanced_features.c, test_char_recursion.c, test_functions.c, test_all_features.c                    |
| float → double varargs 提升 | 呼叫不定參數函式（如 printf）時，自動將 float 純量安插 fnext 提升為 double。                                                | test_fd.c, test_functions.c                                                                               |
| void 回傳防護                 | 處理無回傳值函式，確保在遇到空 return; 或結尾時皆能正確產生 ret void。                                                        | test_advanced_types.c, test_all_features.c, test_local_fp.c test_functions.c                              |
| 陣列退化（Array Decay）         | 當陣列傳遞至函式時，自動將陣列型別 [N x type]* 無縫轉換為指標型                                                              | test_features.c                                                                                           |

|                                                             |                                                                                                                                                                                                                                        |                                                                        |
|-------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------|
|                                                             | 別 <code>type*</code> ，符合 C 語言標準。                                                                                                                                                                                                       |                                                                        |
| 函式指標宣告與初始化、賦值、當參數與動態呼叫 (Function Pointers)                  | 支援進階 C 語言的函式指標語法 (如 <code>int (*fp)(int, int)</code> )。允許變數儲存函式位址、重新賦值，甚至作為參數動態傳遞給其他函式 (Callback 機制)。底層實作突破了 LLVM 嚴格的型別推導與指標降級陷阱，實現了執行期的動態函式呼叫。                                                                                        | <code>test_argc_funcptr.c</code> ,<br><code>test_ptr_features.c</code> |
| 全域函式指標                                                      | 支援全域函式指標的宣告、Null 初始賦值、重新指向與透過暫存器 <code>load</code> 實現的間接呼叫 (Indirect Call)。                                                                                                                                                            | <code>test_global_fp.c</code>                                          |
| 區域與高階函式指標                                                   | 支援區域變數型態的函式指標宣告、初始化、呼叫與動態切換。支援將函式作為參數傳遞、回傳，並能在迴圈或條件式中動態改變指標的，執行間接呼叫 (Indirect Call)。                                                                                                                                                   | <code>test_local_fp.c</code>                                           |
| 命令列參數支援 ( <code>argc</code> 與 <code>argv</code> )           | 突破基礎 <code>main</code> 函式的無參數限制，完美支援 <code>int main(int argc, char *argv[])</code> 的語法解析與底層記憶體配置。在 LLVM IR 階段精準將其映射為 <code>i32</code> 與雙層字元指標 <code>i8**</code> ，使編譯出的執行檔能像真正的系統 CLI 工具一樣，接收並處理來自終端機的外部參數。                             | <code>test_argc_funcptr.c</code>                                       |
| 外部符號宣告與多檔案連結 ( <code>extern / Separate Compilation</code> ) | 突破單一檔案編譯限制，支援 <code>extern</code> 關鍵字宣告外部全域變數與函式。編譯器前端在符號表中標記外部符號，跳過實體的記憶體配置與預設賦值；並於後端發射 LLVM 的 <code>declare</code> 宣告與外部全域變數參照。支援多個 <code>.c</code> 檔案的分離編譯，並可透過 Linker (如 <code>ld</code> 或 <code>clang</code> ) 進行最終的跨檔案符號位址綁定與連結。 | <code>test_multifile.c</code>                                          |

## 七、標準函式庫

| 功能                                                                                                                 | 文件說明（含功能簡述）                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      | 實際狀態 / 測試檔案                                                                                                                                                |
|--------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 標準字串函式庫 <code>string.h</code>                                                                                      | 支援 <code>strlen</code> , <code>strcpy</code> , <code>strcat</code> , <code>strcmp</code> 等標準字串操作之 LLVM 映射。                                                                                                                                                                                                                                                                                                                                                                                       | <code>test_feat4.c</code> , <code>test_feat6.c</code> ,<br><code>test_stdlib_math.c</code> ,<br><code>test_functions.c</code>                              |
| 型別轉換函式庫 <code>stdlib.h</code>                                                                                      | 支援 <code>atoi</code> / <code>atof</code> 等字串轉整數與轉浮點數的執行期標準庫連結。                                                                                                                                                                                                                                                                                                                                                                                                                                   | <code>test_new5.c</code> ,<br><code>test_stdlib_math.c</code> ,<br><code>test_functions.c</code>                                                           |
| 數學運算函式庫 <code>math.h</code>                                                                                        | 對接 <code>sqrt</code> , <code>pow</code> , <code>fabs</code> , <code>floor</code> , <code>fmod</code> , <code>ceil</code> , <code>sin</code> , <code>cos</code> , <code>log</code> 等數學內建函式。                                                                                                                                                                                                                                                                                                       | <code>test_feat6.c</code> , <code>test_new5.c</code> ,<br><code>test_stdlib_math.c</code> ,<br><code>test_functions.c</code> ,<br><code>test_math.c</code> |
| 數學運算函式庫擴充 ( <code>math.h</code> )                                                                                  | 擴充數學支援，除基礎運算外，新增進階捨入 ( <code>lround</code> , <code>llround</code> 對接並回傳 i64、 <code>nearbyint</code> , <code>round</code> , <code>trunc</code> )、指數對數 ( <code>exp</code> , <code>exp2</code> , <code>log2</code> , <code>log10</code> )、三角與雙曲函式 ( <code>tan</code> , <code>asin</code> , <code>acos</code> , <code>atan</code> , <code>atan2</code> , <code>sinh</code> , <code>cosh</code> , <code>tanh</code> ) 及特殊運算 ( <code>cbrt</code> , <code>hypot</code> )。解決 LLVM 浮點數與 64-bit 整數間的类型別映射。 | <code>test_advanced_features2.c</code>                                                                                                                     |
| 進階浮點數數學運算 ( <code>fmin</code> / <code>fmax</code> / <code>fma</code> / <code>fdim</code> / <code>copysign</code> ) | 擴充進階雙精度浮點數 ( <code>double</code> ) 數學支援。包含雙引數極值判斷 ( <code>fmin/fmax</code> )、正差值計算 ( <code>fdim</code> )、符號複製 ( <code>copysign</code> )，以及三引數融合乘加運算 ( <code>fma</code> )，並自動處理底層浮點數型別提升。                                                                                                                                                                                                                                                                                                         | <code>test_new_features_v4.c</code>                                                                                                                        |
| 浮點數狀態判斷 ( <code>isnan</code> / <code>isinf</code> / <code>isfinite</code> / <code>signbit</code> )                 | 實作 IEEE 754 特殊浮點數狀態驗證。底層映射至 glibc 的內部符號 ( <code>__isnan</code> / <code>__isinf</code> / <code>__finite</code> / <code>__signbit</code> )，能判定 NAN、無限大 (INFINITY) 等特殊值並回傳正確的非零布林整數。                                                                                                                                                                                                                                                                                                                | <code>test_new_features_v4.c</code>                                                                                                                        |
| 系統環境與行程控制 ( <code>getenv</code> / <code>system</code> / <code>atexit</code> / <code>_exit</code> )                 | 接軌作業系統底層呼叫。 <code>getenv</code> 支援回傳 i8* 字串指標抓取環境變數； <code>system</code> 可呼叫 shell 命令並回傳 i32 狀態碼； <code>atexit</code> 支援將函式指標 bitcast 至 i8* 傳入註冊清理回調； <code>_exit</code> 實作硬性終止，底層直接發出 <code>unreachable</code> 指令中斷行程。                                                                                                                                                                                                                                                                          | <code>test_new_features_v4.c</code>                                                                                                                        |
| 餘數與絕對值                                                                                                             | 支援整數絕對值 <code>abs</code> 與浮點數餘數 <code>fmod</code> 運算。                                                                                                                                                                                                                                                                                                                                                                                                                                            | <code>test_stdlib_math.c</code> ,<br><code>test_new5.c</code> ,<br><code>test_functions.c</code>                                                           |

|                                                  |                                                                                                                                                                                                                                                                                                                                                                                                    |                                                                                                   |
|--------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| 字元 I/O                                           | 支援標準字元輸入 <code>getchar</code> 與輸出 <code>putchar</code> 操作。                                                                                                                                                                                                                                                                                                                                         | <code>test_feat4.c</code> ,<br><code>test_stdlib_math.c</code> ,<br><code>test_functions.c</code> |
| 格式化輸出                                            | 支援將格式化字串寫入指定記憶體緩衝區之 <code>sprintf</code> 與 <code>snprintf</code> 操作。                                                                                                                                                                                                                                                                                                                               | <code>test_new5.c</code> ,<br><code>test_stdlib_math.c</code> ,<br><code>test_functions.c</code>  |
| <code>printf</code> 跳脫序列                         | 用「反斜線」告訴編譯器：後面的字元不是普通的文字，轉換成特殊指令。<br>Ex: <code>printf("tab:\there\n");</code>                                                                                                                                                                                                                                                                                                                      | <code>test_advanced_types.c</code>                                                                |
| 字串、字元、浮點數輸入支援                                    | 擴充輸入功能， <code>scanf</code> 可直接透過 <code>%s</code> 、 <code>%c</code> 、 <code>%f</code> 讀取連續字元至指定的記憶體陣列。                                                                                                                                                                                                                                                                                              | <code>test_all_features.c</code> ,<br><code>test_new4.c</code> ,<br><code>test_functions.c</code> |
| 進階格式化輸出支援 ( <code>%p</code> / <code>%zu</code> ) | 強化 <code>printf</code> 可變參數 ( <code>vararg</code> ) 處理。攔截 <code>Pointer</code> 型別或 <code>exactTypeMap</code> 記錄之指標，動態 <code>bitcast</code> 至 <code>i8*</code> 傳出，確保 <code>%p</code> 記憶體位址顯示正確；並支援 <code>%zu</code> 格式符號對應 <code>A uint8_t</code> 型別輸出。                                                                                                                                             | <code>test_new_easy.c</code>                                                                      |
| 字元處理函式庫 <code>ctype.h</code>                     | 支援標準 C 語言字元分類與轉換函式 (如 <code>isdigit</code> , <code>isalpha</code> , <code>isalnum</code> , <code>isspace</code> , <code>isupper</code> , <code>islower</code> , <code>isprint</code> , <code>ispunct</code> , <code>isxdigit</code> , <code>isblank</code> , <code>iscntrl</code> , <code>isgraph</code> , <code>toupper</code> , <code>tolower</code> )，並完美對接底層 <code>i32</code> 參數與回傳值，實現精準字元轉換。 | <code>test_final_features.c</code> ,<br><code>test_easy_features.c</code>                         |
| 記憶體與字串操作標準庫                                      | 擴充標準庫支援，實作 <code>memmove</code> (支援重疊記憶體安全複製), <code>memchr</code> , <code>strspn</code> (起始字元集匹配長度), <code>strcspn</code> , <code>strpbrk</code> 等標準 C 字串與記憶體處理函式。                                                                                                                                                                                                                                | <code>test_v6_features.c</code>                                                                   |
| 進階 <code>stdlib</code> API 支援                    | 擴充 C 標準函式庫宣告，新增 <code>labs</code> , <code>llabs</code> , <code>strtoul</code> , <code>strtoul</code> , <code>strtod</code> , <code>strtoll</code> 等進階字串轉數值 API。                                                                                                                                                                                                                                    | <code>test_extended_types_full.c</code>                                                           |
| 執行期斷言防護 <code>assert.h</code>                    | 實作 C 標準庫斷言巨集。在執行期動態評估條件式，若條件為假則自動觸發系統中斷（呼叫 <code>__assert_fail</code> 或 <code>abort</code> ），強化程式的除錯與防呆機制。                                                                                                                                                                                                                                                                                         | <code>test_easy_features.c</code>                                                                 |
| 進階格式化字串輸入                                        | 突破基礎字元輸入限制，支援從字                                                                                                                                                                                                                                                                                                                                                                                    | <code>test_easy_features.c</code>                                                                 |

|                                                               |                                                                                                                                                                                                 |                                                 |
|---------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------|
| (sscanf、fscanf)                                               | 串緩衝區中精準萃取並解析整數 (%d)、浮點數 (%lf) 與字串，並透過 Pass by Reference 將結果寫入變數中。                                                                                                                               |                                                 |
| 單字元串流讀寫 (getc / fputc / ungetc)                               | 擴充單一字元讀寫與推回串流功能。getc 映射至底層 @fgetc；fputc/ungetc 傳入之 char 參數自動觸發 sext (i8→i32) 隱式擴展，符合 C 標準函式庫簽名。                                                                                                 | test_new_easy.c                                 |
| 字串配置複製 (strdup)                                               | 實作 strdup 動態複製字串並回傳 i8*。底層自動將產生的指標登記進 charPtrTemps 與 exactTypeMap 小本本，確保後續讀寫或 free 時的記憶體與型別追蹤安全。                                                                                                | test_new_easy.c                                 |
| string.h 進階擴充函式                                               | 在基礎的 strcpy 之上，擴充實作並註冊更多進階字串處理函式，包含 strncpy, strncat, strncmp, strstr, strchr, strrchr, strtok，大幅提升對 C 語言字串操作的覆蓋率與相容性。                                                                          | test_new4features.c                             |
| 時間與日期處理函式庫 (time.h)                                           | 擴充支援完整的時間處理標準庫。包含 localtime / gmtime (時間結構轉換)、mktime (生成時間戳)、strftime (格式化時間字串) 與 difftime (計算時間差)。編譯器能正確處理 struct tm* 結構體指標的傳遞與存取                                                              | test_advanced_features2.c                       |
| 標準庫亂數函式 (rand / srand)                                        | 擴充 C 標準庫支援。實作「無參數函式呼叫 (如 rand()) 以及「無回傳值 (Void return type)」(如 srand()) 的 AST 解析與 LLVM IR 外部宣告生成。                                                                                               | test_rand.c                                     |
| 檔案 I/O 讀寫與游標控制 (fopen/fclose/fread/fwrite/fseek/ftell/rewind) | 把 C 標準庫的 fopen, fclose 註冊，使編譯器具備讀寫檔案的能力。並擴充二進位檔案讀寫與游標控制能力。fread/fwrite 自動將整數 size/nmemb 進行 sext (i32→i64) 隱式擴充；fseek 支援偏移量自動轉型 i64 並回傳 i32；ftell 回傳 i64 位置；rewind 透過 toFilePtr() 安全轉換 FILE* 防呆。 | test_fileio.c,<br>test_new_easy.c               |
| 進階檔案 I/O (fprintf/fgets/feof)                                 | 擴充支援 fprintf 格式化寫入、fgets 讀取字串，以及 feof 狀態判斷。編                                                                                                                                                    | test_final_features.c,<br>test_easy_features.c, |

|                                                           |                                                                                                                              |                           |
|-----------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|---------------------------|
|                                                           | 譯器底層精確攔截與映射 i8* 指標，確保與 LLVM 完全相容。                                                                                            | test_fileio.c             |
| 進階檔案與行程 I/O 控制<br>(Advanced File & Pipe I/O)              | 實作串流緩衝區的精細控制（clearerr, setbuf, setvbuf）；支援安全暫存檔生成與操作（tmpfile, tmpnam）；並橋接底層 OS，支援雙向 Pipe 行程通訊與外部指令執行（popen, pclose）。         | test_batch1.c             |
| 可變參數格式化 I/O<br>(Variadic Format Output)                   | 支援 stdarg.h 體系的底層字串格式化輸出。包含標準的 vprintf、vfprintf、vsprintf 以及具備緩衝區溢位防護的 vsnprintf。編譯器能正確傳遞 va_list 參數指標，處理未定長度參數的字串渲染與 I/O 導向。 | test_batch1.c             |
| 錯誤處理與檔案系統管理<br>(ferror/perror/strerror/<br>remove/rename) | 支援系統錯誤捕捉與檔案操作。ferror/strerror/perror 精準處理 errno (i32) 與字串 (i8*) 的 LLVM 型別對接；remove/rename 支援傳入檔案路徑字串並回傳執行狀態碼 (i32)。          | test_new_easy.c           |
| 程式終止控制 (exit)                                             | 實作 exit(i32) 系統呼叫，支援強制中斷並設定回傳狀態碼 (如 exit(0), exit(1))，用以處理異常控制流與錯誤。                                                          | test_final_features.c     |
| 排序與二元搜尋演算法<br>(qsort / bsearch)                           | 實作標準庫之通用排序與搜尋演算法。支援函式指標傳遞，允許將使用者自訂的比較函式作為 Callback 傳入，並於底層處理 void* 指標轉型與動態執行期間接呼叫。                                           | test_advanced_features2.c |
| 浮點數底層拆解與操作<br>(Floating-point Manipulation)               | 對接系統 Math 函式庫，滿足科學計算對 IEEE 754 浮點數結構的直接操作需求。支援提取小數與整數部（modf）、拆解尾數與二進位指數（frexp, ilogb），以及高效的指數縮放乘法（ldexp, scalbn）。            | test_batch1.c             |
| 系統例外與訊號處理<br>(Signal Handling)                            | 實作標準的行程通訊與例外攔截機制。支援以 signal 註冊自訂的非同步 Callback 函式，並可利用 raise 於執行期主動觸發作業系統級別的例外訊號（如 SIGINT, SIGTERM），強化                        | test_batch1.c             |



|                                                           |                                                                                                                                                                                                                                                                                                                                                                               |                                       |
|-----------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------|
|                                                           | 程式的例外防禦與生命週期管理。                                                                                                                                                                                                                                                                                                                                                               |                                       |
| LLVM 底層內建函式 (Built-in Intrinsics)                         | 支援 GCC/Clang 體系的 <code>__builtin_*</code> 編譯器內建函式。包含 <code>popcount</code> (位元 1 個數)、 <code>clz</code> (前導零)、 <code>ctz</code> (尾隨零) 與 <code>bswap</code> (位元組反轉)，及其 64-bit 擴充版本 (*l)。直接對接 LLVM 底層的高效硬體指令，可極速實現 <code>log2</code> 取整或判定 2 的幕次等進階演算法。                                                                                                                          | <code>test_builtins.c</code>          |
| 標準函式庫進階擴充 (stdio / stdlib / string)                       | 擴展 C 標準庫 API 覆蓋率。包含：<br>1. <code>stdio</code> : 支援 <code>puts</code> 自動換行輸出與 <code>gets</code> ； <code>fflush</code> 串流緩衝區排空和 <code>freopen</code> 。<br>2. <code>stdlib</code> : 支援 <code>aligned_alloc</code> 記憶體對齊配置與 <code>_Exit</code> 硬性中斷。<br>3. <code>string</code> : 支援 <code>strsep</code> 進階字串切割（相容空欄位保留，超越 <code>strtok</code> ）與 <code>asprintf</code> 動態配置格式化輸出。 | <code>test_v9_features.c</code>       |
| 標準數學結構體與大數運算 ( <code>div_t</code> / <code>ldiv_t</code> ) | 擴充 <code>&lt;stdlib.h&gt;</code> 之基礎數學與結構體支援。於前端精準支援 <code>div_t</code> 與 <code>ldiv_t</code> 結構體定義，並具備 64-bit 超大長整數 (Long) 的除法 (/) 與取餘數 (%) 運算能力。克服 LLVM 嚴格的結構體型別比對限制，對齊底層系統記憶體佈局。                                                                                                                                                                                           | <code>test_missing_types.c</code>     |
| 三角、雙曲與高階科學計算庫 (math.h)                                    | 支援 IEEE 754 科學計算。對接 <code>asin</code> , <code>acos</code> , <code>atan</code> , <code>atan2</code> （反三角與雙引數座標轉換）、 <code>sinh</code> , <code>cosh</code> , <code>tanh</code> （雙曲函式），以及 <code>cbrt</code> （精準立方根）、 <code>hypot</code> （直角三角形斜邊計算）與 <code>logb</code> （提取浮點數之二進位指數）。                                                                                             | <code>test_advanced_math_env.c</code> |
| 多樣化浮點數捨入與型別轉換機制 (math.h)                                  | 實作浮點數捨入控制。包含跨型別轉換的 <code>lround</code> （捨入並回傳 long）與 <code>llround</code> （捨入並回傳 long long）；以及保持浮點型別的 <code>nearbyint</code> 、 <code>round</code> 、 <code>trunc</code> （無條件截斷至整數部）。編譯器發射對應的隱式提升 ( <code>sext</code> ) 指令。                                                                                                                                                     | <code>test_advanced_math_env.c</code> |
| 字串轉浮點數解析 ( <code>strtouf</code> )                         | 具備將複雜字串（含文字後綴）轉換為 32-bit 單精度浮點數 (float) 的能力。前端 AST 能處理 <code>char</code>                                                                                                                                                                                                                                                                                                      | <code>test_advanced_math_env.c</code> |

|                        |                                                                                       |                          |
|------------------------|---------------------------------------------------------------------------------------|--------------------------|
|                        | <b>**endptr</b> 的雙重指標語意，允許函式在執行期動態更新外部指標位址。                                           |                          |
| 行程環境變數動態寫入與控制 (putenv) | 在原有的 <b>getenv</b> 唯讀機制基礎上，新增 <b>putenv</b> 系統呼叫支援。編譯器可動態修改或注入當前行程的環境變數表，實現執行期的環境配置同步 | test_advanced_math_env.c |

## 八、指標與記憶體管理

| 功能                                     | 文件說明（含功能簡述）                                                                                                                                                | 實際狀態 / 測試檔案                                                                                               |
|----------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| 指標系統                                   | 支援 <b>&amp;</b> 取址、 <b>*</b> 解參考、指標型別安全轉換 ( <b>bitcast</b> ) 與解參考賦值。                                                                                       | ( <b>isPointer</b> , <b>pointeeType</b> )                                                                 |
| 指標算術與進階解參考                             | 支援陣列元素精準取址 <b>&amp;arr[idx]</b> 、指標算術 <b>p + n</b> 、解參考寫入 <b>*(expr) = val</b> 。                                                                           | test_pointer.c,<br>test_new5features.c,<br>test_pointer_.c,<br>test_ptr_features.c,<br>test_type_system.c |
| 指標宣告與進階操作                              | 支援完整的指標宣告、取址 ( <b>&amp;</b> )、解參考讀取 ( <b>*</b> )、指標算術 ( <b>p+n</b> ) 及直接賦值 ( <b>*p = val</b> )，並支援指標的宣告時直接初始化、 <b>pass-by-reference</b> 、 <b>float</b> 指標。 | test_pointer.c,<br>test_new5features.c,<br>test_pointer_.c,<br>test_ptr_features.c,<br>test_type_system.c |
| 多重指標                                   | 支援多重指標 <b>int **pp</b> ，讓指標系統更完整。                                                                                                                          | test_mul_pointer.c                                                                                        |
| 指標強制轉型 ( <b>Pointer Type Casting</b> ) | 支援 ( <b>struct Node*</b> ) <b>malloc(...)</b> 或指標間強制轉型，底層透過 <b>bitcast</b> 轉換型別，使原始記憶體具備結構與指標特性。                                                           | test_pointer_struct.c                                                                                     |
| 指標參數傳遞、 <b>float</b> 指標                | 允許將指標作為參數傳遞，實作跨函式變數數值修改（模擬 <b>Pass-by-Reference</b> ）。                                                                                                     | test_pointer_.c,<br>test_pointer.c,<br>test_ptr_features.c,<br>test_local_fp.c                            |
| 函式回傳指標 ( <b>Pointer Return</b> )       | 支援函式回傳 <b>int*</b> 或 <b>struct Node*</b> 等指標型別，並於呼叫端透過 <b>funcRetPointeeRegistry</b> 查表精準接住指標，解決指標退化為 <b>i8*</b> 的問題。                                      | test_new5features.c,<br>test_ptr_features.c                                                               |
| 全域函式指標                                 | 支援全域函式指標的宣告、 <b>Null</b> 初始賦值、重新指向與透過暫存器 <b>load</b> 實現的                                                                                                   | test_global_fp.c                                                                                          |

|                            |                                                                                                                 |                                                                      |
|----------------------------|-----------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------|
|                            | 間接呼叫 (Indirect Call) 語意。                                                                                        |                                                                      |
| 區域與高階函式指標                  | 支援區域變數型態的函式指標宣告與動態切換。支援將函式作為參數傳遞 (First-class functions / 高階函式設計)，並能在迴圈或條件式中動態改變指標標的，執行精準的間接呼叫 (Indirect Call)。 | test_local_fp.c                                                      |
| 動態記憶體配置與釋放 (malloc / free) | 支援動態記憶體配置 malloc 與釋放 free，支援將指標自動轉型並附帶防呆保護機制以防 LLVM 崩潰。                                                         | test_new5features.c,<br>test_pointer_struct.c,<br>test_linked_list.c |
| 動態記憶體配置 (calloc / realloc) | 擴充底層記憶體管理能力。calloc 支援配置記憶體並自動歸零初始化；realloc 支援在保留原有資料的前提下，動態擴充或縮小記憶體區塊，對接底層系統呼叫。                                 | test_easy_features.c                                                 |
| memset、memcpy、memcmp       | 支援底層記憶體區塊的複製、初始化與比較，並內建指標自動轉型防護（如自動轉為 i8*），防止 LLVM 型別衝突。                                                        | test_new5features.c                                                  |
| 箭頭運算子 (->) 的讀取與寫入          | 支援透過箭頭運算子直接讀取或修改結構體指標的內部成員，並計算欄位偏移量。                                                                            | test_final_features.c,<br>test_pointer_struct.c                      |
| 自我參考 (Self-Referential)    | Linked List（鏈結串列）（例如結構體 Node 裡面包著另一個 struct Node* next），可走訪、插入節點。                                               | test_linked_list.c                                                   |
| NULL 指標防護網                 | 在語意分析階段精準攔截常數 0 賦值給指標的操作，並在編譯期自動將其替換為 LLVM 原生的 null，徹底解決 C 語言語意與 LLVM 強型別系統之間的 Null Pointer 衝突。                 | test_features.c                                                      |

## 九、預處理器

| 功能                       | 文件說明（含功能簡述）                        | 實際狀態 / 測試檔案                                                                                |
|--------------------------|------------------------------------|--------------------------------------------------------------------------------------------|
| #define / #undef         | 支援常數巨集（Macro）的建立與替換，並支援動態解除巨集定義。   | test_preprocessor.c,<br>test_func_marco.c,<br>test_preprocessor_.c,<br>test_all_features.c |
| 帶有運算式求值的條件編譯 (#if、#elif) | 預處理器能力大幅升級，從單純的「判斷巨集是否存在」，進化為具備「布林 | test_easy_features.c                                                                       |

|                                                                                                             |                                                                                                                                                                                                                |                                                                                               |
|-------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
|                                                                                                             | 邏輯與數值運算」能力。能直接在編譯期評估如 <code>#if MAX_SIZE &gt; 50</code> 等複雜條件式，精準進行死碼消除與區塊過濾                                                                                                                                   |                                                                                               |
| 函式型巨集                                                                                                       | 支援 <code>#define MAX(a, b) ((a) &gt; (b) ? (a) : (b))</code> 函式型巨集。                                                                                                                                            | test_func_marco.c                                                                             |
| 條件編譯指令                                                                                                      | 支援基於巨集存在與否的條件編譯區塊過濾（ <code>#ifdef</code> , <code>#ifndef</code> , <code>#else</code> , <code>#endif</code> ）。                                                                                                  | test_preprocessor.c,<br>test_preprocessor_.c,<br>test_easy_features.c                         |
| 科學記號與後綴解析                                                                                                   | 詞法分析支援解析帶指數（ <code>e/E</code> ）或精度後綴（ <code>f/F</code> ）的浮點數常數。                                                                                                                                                | test_advanced_types.c,<br>test_advanced_suite.c,<br>test_preprocessor_.c,<br>test_strict_80.c |
| 字串化（ <code>#</code> ）與精準 Token 連接（ <code>##</code> ）                                                        | 實作了 C 語言前置處理器中進階的 <code>##</code> （HASHHASH）運算子，支援符號拼接。升級巨集展開邏輯為嚴格的瀑布流三步驟：1) 處理 <code>#param</code> 將引數包裝為跳脫字面值字串；2) 進行普通變數替換；3) 最終消除 <code>##</code> 實現精準 Token 連接。透過 Negative Lookbehind Regex 防護網避開邊界陷阱與誤判。 | test_new_easy.c,<br>test_advanced_suite.c                                                     |
| 標準內建巨集 (Built-in Macros)                                                                                    | 擴充 Lexer 預處理器，實作 C 語言標準內建巨集替換：包含 <code>__LINE__</code> （展開為當前行號）、 <code>__FILE__</code> （動態取得並展開真實的編譯原始碼檔名）以及 <code>__func__</code> （展開為當前所在的函式名稱）。                                                            | test_new4features.c                                                                           |
| 編譯器指令靜默與警告提示<br>( <code>#pragma</code> / <code>#error</code> / <code>#warning</code> / <code>#line</code> ) | 實作編譯器控制指令解析。靜默忽略 <code>#pragma once</code> 、 <code>#pragma GCC</code> 等優化指令以及 <code>#line N</code> ；遇到 <code>#error "msg"</code> 與 <code>#warning "msg"</code> 時，捕捉字串並於終端機印出對應提示訊息後繼續執行，不干擾編譯流程。               | test_new_features_v4.c                                                                        |
| 動態巨集展開<br>( <code>__COUNTER__</code> / <code>__DATE__</code> / <code>__TIME__</code> )                      | 支援動態環境變數展開。 <code>__COUNTER__</code> 採全域計數器跨迴圈共享，確保每次展開皆遞增生成唯一值； <code>__DATE__</code> 與 <code>__TIME__</code> 於編譯期動態呼叫 <code>LocalDateTime.now()</code> ，實時展開為標準 C 語言字串字面值。                                   | test_new_features_v4.c                                                                        |
| 多行巨集解析 (Multi-line Macros)                                                                                  | 擴充 Preprocessor 功能，支援以反斜線 \ 進行多行程式碼接合。支援 Object-like                                                                                                                                                           | test_multiline_macro.c                                                                        |

|                                                |                                                                                                                                                             |                                 |
|------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------|
|                                                | 與 Function-like 多行巨集宣告，並能與字串化運算子 (#) 及條件編譯指令混用。                                                                                                             |                                 |
| 可變參數巨集與逗號吞噬<br>(__VA_ARGS__ /<br>##_VA_ARGS__) | 於 MacroDef 結構新增 isVariadic 旗標。透過 ... 收集所有不具名引數；並實作進階防呆機制，當 ##__VA_ARGS__ 遇到無傳入引數的狀況時，自動靜默吃掉前方的逗號，消滅語法解析錯誤 (Syntax Error)。                                   | test_new_features_v4.c          |
| 預定義常數直接注入                                      | 於 Preprocess 階段直接將 C 語言核心常數注入符號表。包含 INFINITY、NAN、HUGE_VAL、M_PI、M_E、M_SQRT2、M_LN2，以及 INT_MAX、INT_MIN、CHAR_MAX、SEEK_SET/CUR/END、TRUE/FALSE 等超過 20 個數學與系統邊界常數。 | test_new_features_v4.c          |
| 標頭檔引入與展開<br>(#include)                         | 強化 Preprocessor 功能，支援 #include "... " 自訂標頭檔展開。能在編譯前動態讀取外部檔案並將其 AST 內容遞迴插入當前程式碼，接合外部定義之常數巨集與函式實作，完善多檔案專案的模組化編譯能力。                                            | test_include.c<br>test_helper.h |

## 十、底層架構

| 功能                                       | 文件說明（含功能簡述）                                     | 實際狀態 / 測試檔案                           |
|------------------------------------------|-------------------------------------------------|---------------------------------------|
| 字串常數池去重                                  | 相同字串重複出現時不重複配置記憶體，編譯期統一共用同一個全域指標位址。             | test_arch.c,<br>test_advanced_types.c |
| IR Buffer Stack (pushBuffer / popBuffer) | 利用多層級緩衝區堆疊，隔離與延遲輸出控制流之 Body 指令。                 | test_arch.c                           |
| Terminator Guard                         | 防護機制，確保在 return/break/continue 終止符後不插入多餘的無效 br。 | test_arch.c                           |
| charPtrTemps 指標退化追蹤                      | 追蹤字元陣列退化狀態，防止對 char* 產生多餘或錯誤的讀取（load）。          | test_arch.c                           |
| 十六進位浮點數精準轉換                              | 將浮點常數以 IEEE 754 十六進位形式寫入 IR，確保編譯期間精確度絕不遺失。      | test_arch.c                           |
| ReturnTypeStack                          | 追蹤並驗證巢狀或多重回傳函式的型別一致性，確保返回對應的 LLVM 型別。           | test_arch.c                           |

|                          |                                                                                     |                 |
|--------------------------|-------------------------------------------------------------------------------------|-----------------|
| UTF-8 與非 ASCII 字元支援      | 手刻碼位解析，正確計算中文字元長度並自動轉換為合規之 LLVM \XX 十六進位跳脫序列。                                       | test_features.c |
| 全域型別追蹤小本本 (exactTypeMap) | 記住每一個虛擬暫存器的真實 LLVM 型別（如 %struct.Node* 或 i32*），解決型別退化成 i8* 的問題，讓指標算術與 bitcast 更安全精準。 | test_features.c |

## 十一、詞法與語意安全

| 功能                                      | 文件說明（含功能簡述）                                                                                                                             | 實際狀態 / 測試檔案            |
|-----------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|------------------------|
| 跳退字元全解析                                 | 在處理字串與字元字面值時，精準識別並跳過所有合法的 C 跳退字元干擾。                                                                                                     | test_semantic_safety.c |
| L-value 與 R-value 審查                    | 賦值時精準區分左值（變數記憶體）與右值，嚴格防止對唯讀常數或字面值進行非法寫入。                                                                                                | test_semantic_safety.c |
| 語意分析防護網                                 | 實作全方位防護，包含作用域檢查、型別安全防呆、自動隱式轉型，並具備極致的錯誤恢復（Error Recovery）能力，防止編譯器崩潰。                                                                     | test_semantic_safety.c |
| C11 編譯期靜態斷言 (_Static_assert)            | 支援 C11 標準之靜態斷言機制。結合常數折疊技術，於 AST 解析階段評估編譯期常數條件 (如 sizeof 檢查)，提早攔截架構相依性錯誤。支援全域/區域作用域及無訊息版本；若條件誤傳為執行期變數。                                   | test_static_assert.c   |
| void 顯式強迫轉型與無副作用敘述 (Explicit Void Cast) | 於前端 AST 實作對 (void)expr 的合法轉型支援。允許開發者或巨集使用此 C 語言經典語法來明確忽略運算式回傳值（如抑制未使用變數之警告）。編譯器會靜默賦予 TypeInfo.Void 型別並消除其 LLVM IR 副作用，接軌現代 C 語言的防呆開發規範。 | test_batch1.c          |
| 函式簽名與實作一致性審查                            | 強化函式前置宣告 (Prototype) 與實作定義 (Definition) 之間的語意防禦。攔截「回傳型別不一致」、「參數數量/型別錯置」以及「函式重複實作」等致命錯誤。同時於呼叫端動態驗證傳入之引數數量，並內建可變參數函式 (Variadic            | test_func_check.c      |

|  |                                                          |  |
|--|----------------------------------------------------------|--|
|  | Functions, 如 <code>printf</code> ) 的安全豁免機制，確保跨區塊呼叫的型別安全。 |  |
|--|----------------------------------------------------------|--|